MASTER'S MODULE · AGENTIC AI · WINTER SEMESTER

Memory for Agents

Why the context window is not memory — and how to build something that is.

Unit 06 · Part 1 of 2 · Session 08 · Week 8

Prof. Dr. Hauke Schramm · Department of Computer Science · Kiel University

Today's agenda

1. **Working memory, not memory** volatility, cost, and limits of the context window
2. **A memory taxonomy** short-term vs episodic, semantic, procedural
3. **Managing the window** truncation, compaction, memory files
4. **Policies** what to write, when to read, how to forget
5. **Architecture** memory as a tool vs automatic injection
6. **Memory in the wild** products, stale memories, poisoning, GDPR

Where we are

Unit 06, part 1 — from architecture to state that survives

Last week (S07) — we chose an architecture: plain code first, framework when it earns its keep

The gap — an agent whose message list dies with the process starts every run from zero

This week — what memory is for an agent, where it lives, and who controls it

Next week (S09) — the heavy machinery for one memory type: vector search and RAG

GUIDING QUESTION

What should an agent remember after the process exits — and what must it be allowed to forget?

1

Working memory, not memory

What the context window actually is

The context window is working memory

The LLM is stateless — each API call is a pure function: tokens in, tokens out, nothing kept

The illusion of memory — the runtime replays the full message list on every call

Working memory — the information currently supplied to the model: messages, tool results, scratchpad notes

Two implementations — a volatile in-process list or a checkpointed transcript (S07): persisted, yet still working memory

Long-term memory — selected information stored for later use, written and retrieved by policy (sections 2 to 4)

TERMINOLOGY CONTRACT

Working memory = what the model currently sees. Long-term memory = selected information kept for later. Persistence and memory function are distinct dimensions.

Why the window is not long-term memory

Three structural reasons, not implementation bugs

Volatility

By default the message list dies with the process. Checkpointing it (S07) restores working memory; it does not select what is worth keeping.

Cost

Every token in context is re-sent and re-billed on every call. Carrying history is a recurring expense, not a one-off.

Hard limits

Windows are finite, and effective use degrades before the limit: models attend poorly to the middle (Liu et al., 2024; context rot, Chroma, 2025).

Re-sending is re-paying

A 100k-token context replayed over a 40-step run bills about 4 million input tokens.

Working memory is paid for on every single call. Prompt caching discounts the replay but does not change the principle — and cache entries live minutes to hours, not weeks.

2

A taxonomy of agent memory

Four types, borrowed from cognitive science

Four memory types

The CoALA framing (Sumers et al., 2024), after Tulving (1972)

Short-term (working)

What the model currently sees: messages, tool results, scratchpad. In-process or checkpointed, it is still working memory — section 3.

Episodic

Records of past runs: what was tried, what failed, what the user said last Tuesday. Answers: what happened?

Semantic

Facts and preferences independent of any episode: the user writes in British English; the project targets Python 3.12.

Procedural

Learned how-to: instructions, skills, house rules. In LLM agents this is mostly text the model is given, plus the weights themselves.

The taxonomy at work

Type	Holds	Example	Typical store	Persisted?
Short-term	current task state	this run's tool results and drafts	context window	if checkpointed
Episodic	past runs, events	a previous run's sources and dead ends	log or run database	yes, by design
Semantic	facts, preferences	the user wants APA citations, EU sources	key-value or vector store	yes, by design
Procedural	instructions, skills	always verify quotes before citing	memory file, system prompt	yes, by design

One agent, four stores — each with its own write path, read path and lifetime; persistence is a separate dimension.

The memory lifecycle

Every long-term memory system implements these four steps



- Write and retrieve are policy decisions — section 4 of this deck.
- Injection lands back in working memory: long-term memory is only useful once it re-enters the window.

3

Managing the context window

Truncation, compaction, and structured notes

Truncation vs summarization

Two ways to keep a long run inside the window

Truncation

- drop oldest messages, keep a sliding window
- cut at valid boundaries: never split a tool call from its result
- system prompt and goal are pinned, never dropped
- loses early constraints and decisions silently

Summarization / compaction

- replace old messages with a model-written summary
- keeps decisions and open questions in few tokens
- costs an extra LLM call at each compaction
- lossy; irreversible unless the originals are archived

Compaction and information loss

Summarization is lossy compression — engineer what survives, at valid message boundaries

Trigger — derive the threshold from input limit, output reserve and the largest next observation; 80% is a rule of thumb

Keep — decisions made, constraints, open questions, file paths, error states

Drop — raw tool dumps, superseded drafts; for web pages keep URL, fetch time and hash — live pages change

The trade-off — every compaction is a bet on what the future run needs; treat the summary as a lossy index, keep source snapshots outside the window

Anchors — MemGPT (now Letta) framed this as OS-style paging (Packer et al., 2023); Claude Code ships it as auto-compact

Structured notes: a memory file as two tools

Notes survive compaction and restarts — state worth keeping leaves working memory deliberately

memory_tools.py

```
from pathlib import Path
from datetime import date
MEMORY = Path("memory/notes.md")
MEMORY.parent.mkdir(parents=True,
                    exist_ok=True)

def memory_read() -> str:
    """Notes file, read whole."""
    if not MEMORY.exists(): return ""
    return MEMORY.read_text()

def memory_append(entry: str) -> str:
    """Add one short, dated note."""
    with MEMORY.open("a") as f:
        f.write(f"- {date.today()}: {entry}\n")
    return "stored"
```

Mechanism — a small text file, read at start, appended as the agent works; CLAUDE.md style

Why text — human-readable, versionable in git, portable

Model decides — what is worth a note (Anthropic, 2025); pruning is a separate pass

Not an audit trail — a readable append log; provenance, integrity, access control and concurrent writers need extra design

4

Write, read, forget

The policies that make memory useful

Write policies: what is worth storing

Store — stable, reusable, expensive to re-derive: preferences, decisions, verified facts

Not in active memory — transient detail, raw outputs — the evidence archive and the trace are separate stores

When — inline during the run, or as a consolidation pass at the end of it

Distill first — store the conclusion with provenance, time, confidence and scope, not the transcript

Deduplicate — check for an existing memory to update before adding a near-copy

COST ASYMMETRY

A useless memory is not free: it pollutes every future retrieval and every future prompt.

Read policies: when to retrieve

Session start — load a small, curated core — identity, preferences, house rules

On demand — the agent queries memory when it notices it lacks context

Similarity-triggered — retrieve entries near the current task — next week's machinery

Scoring — a weighted sum of normalized recency, importance and relevance — the Generative Agents recipe (Park et al., 2023)

Relevance risk — an irrelevant retrieved memory actively misleads; weigh the cost of a wrong retrieval against the cost of a missing fact

Forgetting and aging

A memory store without deletion degrades into noise — Lab 08 (R2) implements the first and third

Time-based expiry

entries carry a TTL; volatile facts (current sprint, deadlines) age out automatically.

Usage-based decay

memories never retrieved lose score over time — rarely-used entries sink, frequently-used ones persist.

Supersession

a new fact replaces its contradiction: 'prefers React' overwrites 'prefers Vue', with a timestamp.

Explicit deletion

user-invoked or policy-invoked removal — also a legal requirement, as section 5 will show.

Tool call vs automatic injection

Who decides when memory enters the context?

Memory as a tool

- model calls memory_read / memory_append explicitly
- deliberate, auditable, token-frugal
- adapts retrieval to the task at hand
- but: the model may simply not call it

Automatic injection

- runtime inserts memories into the prompt
- ensures selected items are present, not that selection is correct
- costs tokens on every call, relevant or not
- silent influence — hard to audit, easy to poison

5

Memory in the wild

Products, pitfalls, and the law

Memory in real products

The same design space, shipped at scale

ChatGPT memory — auto-extracted user facts injected into chats; user can view and delete

Claude memory — non-project and project memory; scope and controls depend on product, plan and settings (Anthropic help, 2026)

Project files — CLAUDE.md and similar: procedural memory as a versioned repo artifact

Coding agents — rules files plus auto-compaction: short-term and procedural combined

Common pattern — hybrid — curated injection at start, tool access for the long tail

Pitfalls: stale, poisoned, private

Stale memories — preferences change; a confident wrong memory beats no memory at being harmful

Memory poisoning — untrusted content writes itself into memory: a persistent prompt injection (S13; Lab 08, R3)

Provenance — store who wrote each memory and from what source — trust is not uniform

GDPR Art. 17 — where erasure is required, propagate it to personal-data copies and derived representations; document exceptions and retention duties

Deletion design — track every copy — cache, embeddings, summaries, backups — so a restore cannot resurrect erased data

Design rule — memories from untrusted sources never become instructions

Lab 08: measure what carries over

Local Ollama model (qwen2.5:7b), offline eight-page corpus, gap-fill notebook, Parts A to G, 90 to 120 min

Token meter (B) — a chars/4 estimate reproduces the re-sending cost curve: keep everything vs a window managed at 8k tokens

Truncation vs compaction (C, D) — pinned system prompt and goal vs an 80% trigger with keep/drop lists; a mentions() detector measures what each loses

Memory file (E) — the 14-line pattern: memory_read / memory_append, write policy in the tool description, end-of-run consolidation

Two sessions (F) — Monday battery recycling with standing requirements, Wednesday lithium supply chains without — amnesiac vs memory file

Report R1 to R3 — classify every stored note (CoALA); forgetting pass with TTL and supersession (R2); the planted memory-poisoning page (R3)

TAKEAWAYS

What you should remember

- **Working memory —** what the model currently sees: in-process or checkpointed, re-billed per call, bounded
- **Taxonomy —** working memory plus episodic, semantic, procedural stores; persistence is a separate dimension
- **Window management —** truncate or compact at valid boundaries, or move state into memory files
- **Policies —** write distilled facts with provenance, retrieve by cost of error, forget on schedule
- **Architecture —** tool calls are deliberate and auditable; injection guarantees presence, not correctness
- **Risks —** stale facts, memory poisoning, and the right to be forgotten

NEXT SESSION Vector databases and retrieval-augmented generation.

Reading & references

- Sumers, T., Yao, S., Narasimhan, K., Griffiths, T. (2024). Cognitive Architectures for Language Agents. TMLR. arXiv:2309.02427.
  CoALA: the working/episodic/semantic/procedural framing for LLM agents.
- Tulving, E. (1972). Episodic and Semantic Memory. In: Organization of Memory. Academic Press.
- Packer, C. et al. (2023). MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560.
- Park, J. S. et al. (2023). Generative Agents: Interactive Simulacra of Human Behavior. UIST 2023. arXiv:2304.03442.
  Memory stream; retrieval score: weighted sum of normalized recency, importance and relevance; reflection.
- Liu, N. F. et al. (2024). Lost in the Middle: How Language Models Use Long Contexts. TACL 12.
- Anthropic (2025). Effective Context Engineering for AI Agents. Engineering blog.
- Greshake, K. et al. (2023). Not What You've Signed Up For: Indirect Prompt Injection. arXiv:2302.12173.
- Chroma Research (2025). Context Rot: How Increasing Input Tokens Impacts LLM Performance.
- Anthropic (2026). Use Claude's chat search and memory. Help Center, support.claude.com/en/articles/11817273 (accessed 1 Oct 2026).
- Regulation (EU) 2016/679 (GDPR), Article 17: Right to Erasure.